HeliOps

User Guide

Mission Editor, traffic, routes, tasks, airports and radio

Based on HeliOps_Framework_Template.miz and the current HeliOps code
HeliOps 0.1 - Syria map / Cyprus template
Document date: 2026-09-10

Document scope - This is the user and Mission Editor guide. Installation of SRS, DCS-gRPC, the bridge, external DCS modifications, and version locking are covered in the separate installation guide.

Contents

1. Overview and core principle

HeliOps is a thin framework on top of MOOSE for building reproducible AI traffic, MOOSE FLIGHTCONTROL-based airfield control, and radio control in DCS. MOOSE owns routing, waypoint passage, waypoint tasks, task duration, RTB, holding, and landing. HeliOps should primarily configure and connect these functions - not duplicate them.

The current template mission is built for the Syria map, with Cyprus as the primary operating area. RAF Akrotiri is the HomeBase and default RTB for template traffic. Paphos and Larnaca also have HeliOps FLIGHTCONTROL configuration and dedicated holding zones.

> Important design rule - Prefer changing HeliOps configuration or Mission Editor objects. Do not patch MOOSE for normal HeliOps use. The bundled lib/Moose.lua is version-locked to HeliOps.

2. Recommended workflow: start from the template

The safest way to create a new HeliOps mission is to open HeliOps_Framework_Template.miz in the Mission Editor and save it under a new name. The template already contains MOOSE, the HeliOps Lua files, the trigger load order, trigger zones, client groups, and AI groups expected by HeliOps.

Step	Action
1	Open HeliOps_Framework_Template.miz in the DCS Mission Editor on the Syria map.
2	Use Save As for the new mission. Do not modify the original template.
3	Keep the Lua trigger chain and its order.
4	Move/change trigger zones as needed, but keep their names unless the corresponding Lua is also changed.
5	Keep exact DCS group names, or update HeliOps_ME_Assets.lua and the traffic script at the same time.
6	Change route, task, and profile behavior in Traffic_*.lua - not in HeliOps_ME_Assets.lua.
7	Run a test and check dcs.log for HELIOPS ... START/READY and any SCRIPTING ERROR.

3. Mission Editor - Lua load order

The template mission uses DO SCRIPT FILE actions. MOOSE is loaded at MISSION START. The remaining HeliOps files are loaded by ONCE rules after TIME MORE 1 second, in the order in which they appear in the trigger list. This order is part of the dependency chain and should be treated as a contract.

#	File	ME-trigger	Purpose
1	Moose_.lua	MISSION START	MOOSE core; must exist before HeliOps uses GROUP, FLIGHTGROUP, FLIGHTCONTROL, MSRS, etc.
2	HeliOps.lua	ONCE, TIME > 1 s	Core namespace, HeliOps:RegisterFlight(), utility functions.
3	HeliOps_Debug.lua	ONCE, TIME > 1 s	Debug/event logging.
4	HeliOps_Config.lua	ONCE, TIME > 1 s	Global HeliOps/Cyprus configuration.
5	HeliOps_ME_Assets.lua	ONCE, TIME > 1 s	Single source of truth for group and zone names in ME.
6	HeliOps_Nav.lua	ONCE, TIME > 1 s	Navigation-zone database and route-zone utilities.

#	File	ME-trigger	Purpose
7	HeliOps_Airports.lua	ONCE, TIME > 1 s	Airport/RTB/radio/ holding configuration.
8	HeliOps_FlightControl.lua	ONCE, TIME > 1 s	Creates MOOSE FLIGHTCONTROL for enabled airports.
9	HeliOps_RadioRouter.lua	ONCE, TIME > 1 s	Radio-command router to native MOOSE player callbacks.
10	HeliOps_Traffic.lua	ONCE, TIME > 1 s	Generic traffic framework.
11	Traffic_Fighters.lua	ONCE, TIME > 1 s	Starts fighter traffic.
12	Traffic_Heavy.lua	ONCE, TIME > 1 s	Starts heavy/support traffic.
13	Traffic_Helicopters.lua	ONCE, TIME > 1 s	Starts helicopter traffic.

Why order matters - Example: HeliOps_Airports.lua references HeliOps.ME.Assets.Zones, so HeliOps_ME_Assets.lua must be loaded first. HeliOps_FlightControl.lua requires Airports. Traffic_*.lua requires HeliOps_Traffic.lua and ME assets.

4. Responsibility of each HeliOps file

File	Responsibility	Normally changed when...
HeliOps.lua	Core and registration of an existing DCS group as a MOOSE FLIGHTGROUP.	Framework core changes.
HeliOps_Config.lua	Global debug/verbosity and overall Cyprus defaults.	You want to change global debug/verbosity or the HomeBase default.
HeliOps_ME_Assets.lua	Exact ME names and DCS type IDs for groups/zones.	You add, remove, rename, or change the type of ME assets.
HeliOps_Nav.lua	List of nav zones per area and helper functions.	You create additional operating areas or zone pools.
HeliOps_Airports.lua	Airport key, MOOSE AIRBASE, frequency, holding, and traffic limits.	You add an airport or change radio/holding/capacity.
HeliOps_FlightControl.lua	Generic wrapper around MOOSE FLIGHTCONTROL/MSRS.	Rarely; airport-specific values should normally not be hard-coded here.

File	Responsibility	Normally changed when...
HeliOps_RadioRouter.lua	Connects radio flags to the correct FLIGHTCONTROL and native MOOSE callbacks.	New radio commands or a radio interface are needed.
HeliOps_Traffic.lua	Generic traffic engine: config validation, route, tasks, RTB, landing policy, and damage abort.	New generic traffic functionality is needed.
Traffic_Fighters.lua	Fighter profiles and fighter instances.	Fighter routes/speeds/tasks/profiles change.
Traffic_Heavy.lua	Heavy/support profiles and instances.	Tanker/AWACS/transport routes/speeds/tasks change.
Traffic_Helicopters.lua	Helicopter profiles and instances.	Helicopter routes/speeds/tasks change.

5. Mission Editor assets - names are a contract

HeliOps_ME_Assets.lua is the intended single source of truth for assets that actually exist in the Mission Editor. It should mirror ME, not contain route logic, task logic, or behavior. The DCS Group Name must exactly match Name in this file.

```
HeliOps.ME.Assets.Groups = {
   Viper21 = { Name = "Viper21", Type = "F-16C_50" },
   Hip21   = { Name = "Hip21",   Type = "Mi-8MT" },
   Texaco21= { Name = "Texaco21",Type = "KC-135" },
}
```

Group Name vs Unit Name - HeliOps traffic primarily identifies the DCS GROUP by Group Name. Unit names may follow the group name, e.g. Hip21-1 and Hip21-2, but do not change Group Name without also changing the ME asset registry and traffic scripts that reference the group.

6. AI groups in the template

Category	DCS Group Name	Type	Units	Skill
Fighter	Viper21	F-16C bl.50 / F-16C_50	2	High
Fighter	Viper22	F-16C bl.50 / F-16C_50	2	High
Fighter	Hornet21	F/A-18C / FA-18C_hornet	2	High
Fighter	Hornet22	F/A-18C /	2	High

Category	DCS Group Name	Type	Units	Skill
		FA-18C_hornet		
Fighter	Tomcat21	F-14B(U) / F-14B	2	High
Fighter	Tomcat22	F-14B(U) / F-14B	2	High
Helicopter	Hip21	Mi-8MTV2 / Mi-8MT	2	High
Helicopter	Hip22	Mi-8MTV2 / Mi-8MT	2	High
Helicopter	Apache21	AH-64D BLK.II	2	High
Helicopter	Chinook21	CH-47F	2	High
Heavy/support	Texaco21	KC-135	2	High
Heavy/support	Texaco22	KC-135	2	High
Heavy/support	Magic21	E-3A	1	High
Heavy/support	Herc21	C-130J-30	2	High

The template groups are placed as normal AI groups, and the standard traffic scripts register them immediately with HeliOps:RegisterFlight() when Traffic_*.lua is loaded.

7. Late Activation, AI, and start state

In the current template mission, standard HeliOps AI traffic is not implemented as Late Activation traffic. HeliOps_Traffic:Start() finds an already existing DCS group, creates a MOOSE FLIGHTGROUP, builds its MOOSE route, and then waits for the Takeoff event before calling UpdateRoute(). There is no generic ActivateGroup function in the current traffic start sequence.

> Template default - Keep Late Activation OFF for groups started directly by Traffic_Fighters.lua, Traffic_Heavy.lua, or Traffic_Helicopters.lua. If you want to use Late Activation, use a separate explicit activation strategy and test it against the FLIGHTGROUP start flow before making it the default.

The ME start type may be runway, hot parking, or another normal DCS start. The template uses a mixture. HeliOps does not replace the initial ME placement; the framework builds the route and lets MOOSE/DCS manage the flight after departure.

8. Player/Client - Scout and OH-58D

The template contains OH-58D client groups for a human pilot. The primary radio/ATC group is Scout. Scout-1 and Scout-2 are also present as separate Client groups. Skill is Client, not the AI skill High.

Group	Unit	Type	Skill	Template start
Scout	Scout	OH58D	Client	From Parking Area

Group	Unit	Type	Skill	Template start
				Hot, Akrotiri; group radio 251.700 MHz
Scout-1	Scout-1-1	OH58D	Client	From Parking Area Hot
Scout-2	Scout-2-1	OH58D	Client	From Parking Area Hot

RadioRouter no longer registers Scout automatically at mission start. The human group is created as a MOOSE FLIGHTGROUP on the first active HeliOps radio command, for example RADIO_CHECK or TAXI. The same group can then be tracked by MOOSE through taxi, takeoff, and inbound states.

Player vs Client - In a normal single-player mission, DCS Player or Client can be used depending on mission design. The current template uses Client for the Scout groups, which also suits multiplayer/multicrew. HeliOps RadioRouter looks for the active human player unit through the MOOSE database.

9. Trigger zones

All HeliOps zones in the template are circular Mission Editor trigger zones. For route points, the name is more important than the radius; routing uses the zone center as the waypoint coordinate. Holding also uses the center as position 0 for the HeliOps-created holding pattern.

Zone name	Role	Template radius	Use
NAV_CYP_01	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_02	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_03	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_04	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_05	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_06	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_07	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_08	Navigation waypoint	304.8 m (1000 ft)	Center is used as the

Zone name	Role	Template radius	Use
			waypoint coordinate
NAV_CYP_09	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
NAV_CYP_10	Navigation waypoint	304.8 m (1000 ft)	Center is used as the waypoint coordinate
HOLD_AKROTIRI	Holding	3000 m	Center = holding position 0; 3 NM racetrack in the current config
HOLD_PAPHOS	Holding	3000 m	Center = holding position 0; 5 NM racetrack
HOLD_LARNACA	Holding	3000 m	Center = holding position 0; 5 NM racetrack

Move zones - avoid unnecessary renaming - You can move NAV_CYP_xx and HOLD_xx in ME without changing Lua as long as the names remain unchanged. If you rename a zone, HeliOps_ME_Assets.lua and all dependent profiles must be updated.

10. Navigation, NavPool, and Route

HeliOps separates the physical ME layer from flight behavior. NAV_CYP_01..10 are physical zones. An aircraft profile defines a NavPool with waypoint name, speed, and altitude. Route is then a list of indices into that specific NavPool.

```
NavPool = {
   { WP = Zones.NavCyp01.Name, Speed_kt = 250, Alt_ft_msl = 3000 }, -- index 1
   { WP = Zones.NavCyp02.Name, Speed_kt = 280, Alt_ft_msl = 3000 }, -- index 2
   { WP = Zones.NavCyp03.Name, Speed_kt = 500, Alt_ft_msl = 3000 }, -- index 3
   { WP = Zones.NavCyp04.Name, Speed_kt = 500, Alt_ft_msl = 3000 }, -- index 4
   { WP = Zones.NavCyp05.Name, Speed_kt = 250, Alt_ft_msl = 3000 }, -- index 5
}

Route = { 3, 5 }   -- fly NAV_CYP_03 -> NAV_CYP_05
```

Route indices start at 1. A value of 3 therefore means NavPool[3], not automatically NAV_CYP_03; in the template, the index and suffix happen to correspond for the first five points.

10.1 RandomizeRoute

If RandomizeRoute = true, the route plan is shuffled. HeliOps first builds waypoint-index/task-index pairs and shuffles each pair as a unit. Each task therefore remains attached to its intended waypoint even when route order is randomized.

10.2 Speed and altitude limits

Each profile has Limits. If waypoint speed is below MinSpeed_kt or above MaxSpeed_kt, HeliOps corrects the value. Alt_ft_msl is compared with terrain height at the waypoint plus MinTerrainClearance_ft; an MSL altitude that is too low is automatically raised to the minimum permitted value.

> Altitude definition - Alt_ft_msl is feet MSL. MinTerrainClearance_ft is clearance above local terrain at the waypoint zone center. This is a point check at the waypoint, not a full terrain-following route analysis between waypoints.

11. Tasks

Tasks is a list parallel to Route. If Tasks is supplied, it must have the same length as Route. 0 means no task. A positive value references TaskPool[index]. The current HeliOps_Traffic.lua implements ORBIT_CIRCLE as a task type.

```
TaskPool = {
  { Type = "ORBIT_CIRCLE", Duration_s = 120 }, -- 1
  { Type = "ORBIT_CIRCLE", Duration_s = 300 }, -- 2
  { Type = "ORBIT_CIRCLE", Duration_s = 600 }, -- 3
}

Route = { 3, 4, 5 }
Tasks = { 0, 1, 0 } -- 120 s circle orbit at Route[2] / NavPool[4]
```

ORBIT_CIRCLE is built with the MOOSE/DCS TaskOrbit at the waypoint coordinate, altitude, and speed. HeliOps attaches the task to the waypoint with Flight:AddTaskWaypoint(...), including Duration_s.

> Validation - If Route and Tasks have different lengths, HeliOps stops flight startup and logs "Route and Tasks must have same length". A task index that does not exist in TaskPool is also a configuration error.

12. Traffic lifecycle

Phase	What HeliOps does
Config validation	Checks Group, NavPool, Route, Tasks length, and RTB.
Register	HeliOps:RegisterFlight(Group) -> GROUP:FindByName -> FLIGHTGROUP:New.
Landing policy	If TacticalLanding=true, the MOOSE option is set to Overhead Break or Straight In.
Cruise altitude	SetDefaultAltitude(CruiseAlt_ft), in feet.
BuildRoute	Creates MOOSE waypoints and waypoint tasks

Phase	What HeliOps does
	immediately, but does not run UpdateRoute yet.
Takeoff event	OnAfterTakeoff calls UpdateRoute(); the HeliOps/MOOSE route is then sent to DCS.
Enroute	MOOSE handles routing, waypoint passage, and timed waypoint tasks.
Damage abort	If AbortConditions.OnDamage=true, the damaged flight/element proceeds to RTB.
RTB/landing	MOOSE handles RTB/holding/landing according to flight and FLIGHTCONTROL state.

BuildRoute occurring before takeoff while UpdateRoute is called only from OnAfterTakeoff is intentional. Framework comments state that this avoids a false ARRIVED state at mission start.

13. Fighter traffic

Traffic_Fighters.lua defines a shared FighterTaskPool, three profiles, and six group instances. All fighter profiles use TacticalLanding=true with HeliOps.LandingMode.OVERHEAD_BREAK. The default RTB is Akrotiri.

Profile	Min clr ft	Speed kt min-max	Cruise ft	NavPool speed/alt (1..5)
F16C	300	180-500	3000	250/3000; 280/3000; 500/3000; 500/3000; 250/3000
F18C	100	180-550	3000	250/3000; 280/3000; 550/3000; 250/3000; 200/3000
F14B	300	180-550	3000	250/3000; 280/3000; 500/3000; 500/3000; 250/3000

Group	Profile	Default Route	Default Tasks	RTB
Viper21	F16C	{3,5}	{0,0}	Akrotiri
Viper22	F16C	{3,5}	{0,0}	Akrotiri
Hornet21	F18C	{3,5}	{0,0}	Akrotiri
Hornet22	F18C	{3,5}	{0,0}	Akrotiri
Tomcat21	F14B	{3}	{0}	Akrotiri
Tomcat22	F14B	{3}	{0}	Akrotiri

14. Helicopter traffic

Traffic_Helicopters.lua uses the same general HeliOps.Traffic engine but separate rotorcraft profiles. TacticalLanding is not set in these profiles, so native MOOSE/DCS landing behavior is left unchanged.

Profile	Min clr ft	Speed kt min-max	Cruise ft	NavPool speed/alt (1..5)
MI8	100	30-140	1000	100/1500; 110/1000; 90/1000; 105/1000; 110/1000
AH64D	100	30-160	1000	110/1200; 120/1000; 110/1000; 120/1000; 110/1000
CH47F	100	30-170	1200	120/1500; 130/1200; 120/1200; 130/1200; 120/1200

Group	Profile	Default Route	Default Tasks	RTB
Hip21	MI8	{5}	{0}	Akrotiri
Hip22	MI8	{5}	{0}	Akrotiri
Apache21	AH64D	{4,5}	{0,0}	Akrotiri
Chinook21	CH47F	{4,5}	{0,0}	Akrotiri

15. Heavy / support traffic

Traffic_Heavy.lua covers KC-135 tankers, E-3A AWACS, and C-130J transport aircraft. They use the same route/task/RTB engine but with higher cruise altitudes and profile-specific speed/terrain limits.

Profile	Min clr ft	Speed kt min-max	Cruise ft	NavPool speed/alt (1..5)
KC135	500	180-450	12000	280/10000; 300/12000; 320/12000; 320/12000; 280/10000
E3A	1000	180-430	25000	300/20000; 320/25000; 330/25000; 330/25000; 300/20000
C130J	300	120-320	10000	220/8000; 240/10000;

Profile	Min clr ft	Speed kt min-max	Cruise ft	NavPool speed/alt (1..5)
				250/10000; 250/10000; 220/8000

Group	Profile	Default Route	Default Tasks	RTB
Texaco21	KC135	{3}	{0}	Akrotiri
Texaco22	KC135	{3}	{0}	Akrotiri
Magic21	E3A	{2,4}	{0,0}	Akrotiri
Herc21	C130J	{3,5}	{0,0}	Akrotiri

16. Adding a new AI flight

For a new flight, keep the ME asset, profile, and instance separate. Use this sequence: create the group in the Mission Editor, register it in HeliOps_ME_Assets.lua, select/create a profile in the appropriate Traffic_*.lua, and start it with HeliOps.Traffic:Start().

ME field	Recommendation
Coalition/country	Place under the correct coalition. Template traffic is blue.
Category	Airplane or Helicopter according to type.
Group Name	Unique and exact; this is HeliOps' primary identifier.
Unit Name	Unique names; GroupName-1, GroupName-2, etc. is recommended.
Skill	AI: High in the current template. Human: Client for Scout.
Late Activation	OFF for the standard Traffic_*.lua startup flow.
Start position	Place the flight where it should actually start; HeliOps uses the ME start.
ME route	Can be kept minimal. HeliOps builds the operational MOOSE route from trigger zones.

```
-- HeliOps_ME_Assets.lua
NewFlight21 = {
   Name = "NewFlight21",
   Type = "<DCS type id>",
}

-- Traffic_*.lua
```

```
local NewFlight21 = HeliOps.Traffic:Start({
    Group = Groups.NewFlight21.Name,
    Limits = { MinTerrainClearance_ft=100, MinSpeed_kt=100, MaxSpeed_kt=300 },
    CruiseAlt_ft = 3000,
    NavPool = { ... },
    TaskPool = { ... },
    Route = { 3, 5 },
    Tasks = { 0, 0 },
    RandomizeRoute = false,
    AbortConditions = { OnDamage = false },
    RTB = AIRBASE.Syria.Akrotiri,
})
```

17. Airports and FLIGHTCONTROL

HeliOps_Airports.lua is the airport database. Each airport key points to a MOOSE AIRBASE, an RTB base, a FlightControl configuration, and a Hold configuration. HeliOps_FlightControl:CreateAll() creates a controller for each enabled airport.

Airport key	MOOSE airbase	Frequency	Mod	Taxi	Landing	T/O+land	Interval	Hold zone	Hold length
AKROTIRI	AIRBASE.Syria.Akrotiri	251.700 MHz	AM	2	5	2	45 s	HOLD_AKROTIRI	3 NM
PAPHOS	AIRBASE.Syria.Paphos	251.800 MHz	AM	1	default*	default*	default*	HOLD_PAPHOS	5 NM
LARNACA	AIRBASE.Syria.Larnaca	251.850 MHz	AM	1	default*	default*	default*	HOLD_LARNACA	5 NM

* If a value is missing from airport config, the wrapper uses its defaults: LandingLimit 2, LandingTakeoffLimit 0, and LandingInterval 180 s.

17.1 Adding an airport

```
HeliOps.Airports.NEWBASE = {
    Enabled = true,
    Airbase = AIRBASE.Syria.<Name>,
    RTB     = AIRBASE.Syria.<Name>,
    FlightControl = {
        Enabled = true,
        Frequency = 250.000,
        Modulation = radio.modulation.AM,
        TaxiLimit = 1,
        RadioOnlyIfPlayers = true,
        ShowHoldingPatterns = true,
    },
    Hold = {
        Zone = "HOLD_NEWBASE",
```

```
        Length_NM = 5,
        Heading_deg = nil,
        FL_Min = nil,
        FL_Max = nil,
        Priority = 10,
    },
}
```

You must also create HOLD_NEWBASE in the Mission Editor and add the zone to HeliOps_ME_Assets.lua if you want to follow the template structure.

18. Holding patterns

MOOSE FLIGHTCONTROL first creates its backup holding pattern. When Heading_deg, FL_Min, or FL_Max is nil, HeliOps reads heading and flight levels from that backup pattern. HeliOps then creates a new holding pattern with position 0 at the center of the configured ME trigger zone. The MOOSE backup is removed only after the custom pattern has been created successfully.

Parameter	Meaning
Zone	ME trigger zone whose center becomes holding position 0.
Length_NM	Racetrack length.
Heading_deg	If nil: inherit the MOOSE backup heading.
FL_Min / FL_Max	If nil: inherit backup angels; fallback 5/15.
Priority	Holding-pattern priority; template value 10.
ShowHoldingPatterns	Controls MOOSE holding-pattern markings/graphics.

Failsafe - If the holding zone is missing or the custom holding cannot be created, the MOOSE backup is retained instead of leaving the controller without holding.

19. Radio and voice commands

The HeliOps radio chain consists of cockpit radio export, the resident bridge, DCS-gRPC user flags, RadioRouter, and MOOSE FLIGHTCONTROL/MSRS. The spoken station selects the target airport/controller; the actual cockpit frequency validates that the radio is really tuned to that station.

Command key	Type	MOOSE function / HeliOps action
RADIO_CHECK	HeliOps-specific	Tower reply via FLIGHTCONTROL:TransmissionTower().
TAXI	Native MOOSE	_PlayerRequestTaxi

Command key	Type	MOOSE function / HeliOps action
TAKEOFF	Native MOOSE	_PlayerRequestTakeoff
INBOUND	Native MOOSE	_PlayerRequestInbound
DIRECT	Native MOOSE	_PlayerRequestDirectLanding
HOLDING	Native MOOSE	_PlayerHolding
CONFIRM_LANDING	Native MOOSE	_PlayerConfirmLanding

FrequencyToleranceHz is 500 Hz. An incorrect frequency or modulation prevents dispatch to the selected controller function.

Examples of VoiceAttack/bridge arguments:
AKROTIRI RADIO_CHECK
AKROTIRI TAXI
AKROTIRI TAKEOFF
AKROTIRI INBOUND
AKROTIRI DIRECT
AKROTIRI HOLDING
AKROTIRI CONFIRM_LANDING

Station	UHF frequency	Modulation
Akrotiri	251.700 MHz	AM
Paphos	251.800 MHz	AM
Larnaca	251.850 MHz	AM

19.1 Recommended radio sequence for Scout

Phase	Spoken phrase / intent
On the ramp	"Akrotiri, radio check" - registers Scout on the first active HeliOps call and tests the complete radio/TTS chain.
Before taxi	"Akrotiri, request taxi".
Before takeoff	"Akrotiri, request takeoff".
Return	"Akrotiri inbound".
As required	Direct / Holding / Confirm landing according to MOOSE FLIGHTCONTROL state.

No background registration - Scout is not registered at mission start. The first active HeliOps radio command calls HeliOps:RegisterFlight(); if the group has just been created, RadioRouter waits 1.0 s before executing the command so MOOSE can synchronize the FLIGHTGROUP.

20. Radio - what the Mission Editor user needs to know

The OH-58D export reads the AN/ARC-164 UHF radio from cockpit device 30. Template group Scout has a group frequency of 251.700 MHz and the first UHF preset at 251.7. HeliOps routing, however, uses the exported current cockpit state rather than relying only on the ME group frequency.

MOOSE MSRS uses the gRPC backend for both tower and pilot. The tower voice is female/en-GB and the pilot voice is male/en-US. This is the configuration used by the current template to avoid the multi-second simulator freezes previously seen with external executable TTS.

> Installation is separate - SRS, DCS-gRPC, Export.lua, MissionScripting.lua, and HeliOpsBridge are installed according to the installation guide. This User Guide describes how the mission uses them, not the Windows installation itself.

21. Debugging and logging

The template HeliOps_Config.lua uses Debug=true and Verbosity=3. This is useful during development because traffic routes, generated waypoints, FLIGHTGROUP state, and FlightControl startup become visible in dcs.log and sometimes on screen. For finished missions, Debug/Verbosity can be reduced to limit log noise.

Log text	Meaning
=== HELIOPS CORE READY ===	HeliOps core loaded.
=== HELIOPS ME ASSETS READY ===	Asset registry loaded.
=== HELIOPS FLIGHTCONTROL READY ===	Airport controllers created/started.
=== HELIOPS RADIO ROUTER READY ===	Flag polling/radio router active.
HELIOPS: FLIGHTGROUP created: <group>	AI or player group registered as a MOOSE FLIGHTGROUP.
HELIOPS TRAFFIC: <group> ready	Route/profile built and event handlers connected.
Frequency mismatch	Spoken station and tuned cockpit frequency do not match.
Route and Tasks must have same length	Configuration error in Traffic_*.lua.
NAV zone not found	ME trigger zone is missing or has the wrong name.
RTB base not found	RTB AIRBASE name is incorrect or unavailable.

22. Practical mission changes

22.1 Change a route without moving zones

Change the Route list in the appropriate StartFighter/StartHelicopter/StartHeavy call. Example: {3,5} -> {3,4,5}. If Tasks is used, the Tasks list must have the same length.

22.2 Move a waypoint geographically

Move the NAV_CYP_xx zone in the Mission Editor. Keep the name unchanged. All profiles using that zone automatically receive the new geographic point.

22.3 Change speed/altitude for an aircraft type

Change the corresponding profile NavPool in Traffic_Fighters.lua, Traffic_Helicopters.lua, or Traffic_Heavy.lua. This affects all group instances using that profile.

22.4 Change only one flight

Create a separate profile or build a Config specifically for that group. Avoid adding special cases to HeliOps_Traffic.lua when the behavior applies only to one mission/flight.

22.5 Disable a flight

Remove/comment out its StartFighter/StartHelicopter/StartHeavy call, or use a mission-specific enable flag in the traffic script. If the group remains in ME but is never registered by Traffic_*.lua, it does not become a HeliOps-managed traffic flight.

23. Common errors

Symptom	Likely cause	Action
Group not found	ME Group Name and HeliOps_ME_Assets/Traffic config differ.	Compare exact spelling, including hyphens and digits.
NAV zone not found	Trigger zone is missing/has the wrong name.	Create the zone or correct HeliOps_ME_Assets.lua.
Holding zone not found	HOLD_xxx is missing.	Create/move the zone; the MOOSE backup is retained until then.
Route/Tasks length error	Different numbers of entries.	Make the lists the same length.
Unsupported task type	TaskPool contains a type that HeliOps_Traffic does not implement.	Use ORBIT_CIRCLE or implement a new generic task in the framework.
Cannot find flight group Scout	The player has not been registered as a FLIGHTGROUP.	Use RADIO_CHECK/TAXI and check the RadioRouter log.
Negative, must be AIRBORNE to call INBOUND	The MOOSE player FLIGHTGROUP does not have airborne state when called.	Check player registration and takeoff state in the log.

Symptom	Likely cause	Action
No radio response	Incorrect tuned frequency/modulation, bridge/SRS/gRPC inactive, or incorrect station key.	Start with RADIO_CHECK on the correct UHF frequency.
AI does not follow the HeliOps route after takeoff	Takeoff event/UpdateRoute did not occur or the flight was registered incorrectly.	Check TAKEOFF -> UpdateRoute in dcs.log.

24. Mission build checklist

☐ Use the Syria map when the current AIRBASE.Syria constants and Cyprus template are used.

☐ Use Save As to copy the template; do not overwrite the original.

☐ MOOSE + 12 HeliOps/traffic Lua actions are present and in the correct order.

☐ HeliOps_ME_Assets.lua mirrors the actual ME group/zone names.

☐ NAV_CYP_01..10 exist if referenced by profiles.

☐ HOLD_AKROTIRI, HOLD_PAPHOS, and HOLD_LARNACA exist when their respective airports are enabled.

☐ AI Group Names match the traffic scripts.

☐ Standard AI groups have Late Activation OFF unless separate activation logic is used.

☐ Scout is a Client group and exists as an active human group when radio is used.

☐ Route and Tasks have the same length.

☐ All Route indices exist in NavPool.

☐ All non-zero Tasks indices exist in TaskPool.

☐ RTB points to a valid AIRBASE.Syria.*.

☐ Akrotiri/Paphos/Larnaca frequencies match the radio/VoiceAttack configuration.

☐ The mission starts without SCRIPTING ERROR.

☐ RADIO_CHECK, TAXI, TAKEOFF, and INBOUND are tested after major changes.

25. Extend HeliOps without creating parallel functionality

When extending HeliOps, use this order: first check whether MOOSE already provides the function, then use the MOOSE API/FSM/events, and add only a thin HeliOps configuration layer on top. This applies especially to ATC, routing, landing, orbit/tasks, and FLIGHTGROUP state.

Need	Recommended location
New aircraft/helicopter/heavy profile	Traffic_Fighters/Helicopters/Heavy.lua
New group in ME	HeliOps_ME_Assets.lua + the appropriate Traffic_*.lua
New nav zone	ME + HeliOps_ME_Assets.lua + HeliOps_Nav.lua for a new area
New airport	ME holding zone + HeliOps_ME_Assets + HeliOps_Airports
New waypoint task type	HeliOps_Traffic:AddWaypointTask(), preferably built on the MOOSE task API
New radio command already supported by MOOSE	RadioRouter mapping to the native FLIGHTCONTROL callback
Radio feature not provided by MOOSE	Thin HeliOps action, e.g. RADIO_CHECK via MOOSE TransmissionTower

Appendix A. Current template inventory

Category	Names
Player/client groups	Scout, Scout-1, Scout-2
Fighters	Viper21, Viper22, Hornet21, Hornet22, Tomcat21, Tomcat22
Helicopters	Hip21, Hip22, Apache21, Chinook21
Heavy/support	Texaco21, Texaco22, Magic21, Herc21
Holding zones	HOLD_AKROTIRI, HOLD_PAPHOS, HOLD_LARNACA
Navigation zones	NAV_CYP_01 ... NAV_CYP_10
Airports	AKROTIRI, PAPHOS, LARNACA
Radio commands	RADIO_CHECK, TAXI, TAKEOFF, INBOUND, DIRECT, HOLDING, CONFIRM_LANDING

Appendix B. Current tested software context

Component	Tested version / role
DCS World	2.9.29.27278 MT, Windows
MOOSE	commit c48a7d4400161ef6f90967878f2b125ebe7f52ab; bundled under lib/Moose.lua
MOOSE FLIGHTCONTROL	0.7.7

Component	Tested version / role
MOOSE MSRS	0.3.6
DCS-SimpleRadio-Standalone	2.4.0.0
DCS-gRPC	0.8.1
HeliOps RadioBridge	.NET 8 target

For installation details, external file modifications, and version policy, see HeliOps_Installation_Dependencies_and_External_Modifications.pdf.

Appendix C. Minimal traffic config reference

```
local Config = {
    Group = "Example21",

    Limits = {
        MinTerrainClearance_ft = 100,
        MinSpeed_kt = 100,
        MaxSpeed_kt = 300,
    },

    CruiseAlt_ft = 3000,

    NavPool = {
        { WP = "NAV_CYP_01", Speed_kt = 200, Alt_ft_msl = 3000 },
        { WP = "NAV_CYP_03", Speed_kt = 220, Alt_ft_msl = 4000 },
    },

    TaskPool = {
        { Type = "ORBIT_CIRCLE", Duration_s = 120 },
    },

    Route = { 1, 2 },
    Tasks = { 0, 1 },
    RandomizeRoute = false,

    AbortConditions = { OnDamage = false },
    RTB = AIRBASE.Syria.Akrotiri,
}

local flight = HeliOps.Traffic:Start(Config)
```

Appendix D. Source basis and version note

This guide is written against the contents of the supplied HeliOps_Framework_Template.miz. The template contains the Lua files and ME assets described here. Where this document describes current behavior - for example load order, group inventory, zone names, route/task logic, airport frequencies, and radio

commands - it refers to this template snapshot. After later code changes, update the User Guide together with the template and the GitHub repository.